

저전력·신뢰성 완전이해 — 메모리 설계자의 관점

정독용 심층 해설서. F_lowpower.md(저전력 기법)와 직무 질문은행 D1~D8(저전력·신뢰성)을 1st-principles로, 훨씬 깊고 서사적으로 확장한다. 핵심 서사: 누설(leakage)이 retention을 정하고, retention이 refresh를 정하고, refresh와 RowHammer·ECC·aging이 모두 "제어 디지털 로직 + 타이밍 검증"으로 수렴한다. 그 수렴점이 곧 삼성 S.LSI에서 STA·CDC·검증을 하던 지원자의 자리다.

0. 왜 이 문서를 정독해야 하는가 — 한 장의 지도

저전력과 신뢰성은 면접 질문은행에서 별개의 두 카테고리처럼 보이지만, 메모리 설계에서는 사실상 같은 물리 현상의 양면이다. 둘을 잇는 한 단어가 누설(leakage)이다.

- 누설이 크면 → 전력을 더 먹는다(저전력 문제).
- 똑같은 누설이 크면 → DRAM 셀 커패시터의 전하가 빨리 빠진다 → retention time이 짧아진다 → refresh를 더 자주 해야 한다 → 또 전력을 먹는다(신뢰성 ↔ 저전력의 순환).
- refresh가 늦거나 부족하면 → 데이터가 깨진다(신뢰성 문제).
- 미세화로 누설이 커지면 → single-bit error가 늘어난다 → on-die ECC가 필요해진다(신뢰성 문제).

이 문서는 이 사슬을 따라간다. 전력 성분의 물리(1·2장) → 동적전력을 줄이는 레버(3장) → 누설전력을 줄이는 레버(4장) → 메모리 특유의 standby 저전력(5장) → 그 누설이 신뢰성으로 번지는 세 갈래, RowHammer·NAND disturb·on-die ECC(6·7·8장) → 시간이 갉아먹는 소자 aging(9장) → 전부를 하나로 묶는 한 문장(10장) → 면접 골격(11장) → 스스로 점검(12장).

1. 도입 — 메모리에서 전력은 왜 "핵심 스펙"인가

로직 SoC에서 전력은 발열·배터리·전기요금의 문제다. 메모리에서는 한 단계 더 들어간다. 전력이 기능 그 자체와 얽혀 있기 때문이다. DRAM은 가만히 있어도 데이터를 유지하기 위해 refresh라는 전력을 써야 하고, 그 전력을 줄이려다 보면 데이터를 잃을 위험이 커진다. 즉 메모리에서 전력은 "성능을 위해 쓰는 비용"이 아니라 "데이터를 살아 있게 유지하는 비용"이다.

세 가지 무대를 보면 왜 전력이 1순위 스펙인지 분명해진다.

(1) 모바일 — LPDDR. 스마트폰의 DRAM은 대부분의 시간을 idle 상태로 보낸다. 사용자가 화면을 보지 않을 때도 메모리는 데이터를 유지해야 한다. 이때 흐르는 것이 standby/retention 전력이다. 활성 동작 전력(read/write)이 아무리 효율적이어도, 하루 종일 켜져 있는 standby 전력이 배터리를 갉아먹는다. 그래서 LPDDR(Low Power DDR)은 전압을 공격적으로 낮추고(예: LPDDR5는 코어 ~1.05V·변동), self-refresh·deep power-down 같은 저전력 모드를 다단계로 둔다. 모바일 메모리에서 "저전력"은 마케팅 수식어가 아니라 셀 이름(LP)에 박힌 1차 스펙이다.

(2) HBM — 열밀도(thermal density). HBM은 DRAM die를 8~16단 수직으로 쌓고 TSV로 관통 연결한다. 전력은 결국 열로 바뀌는데, 같은 전력을 좁은 부피에 쌓으면 단위 부피당 발열(열밀도)이 폭증한다. 특히 적층

한가운데 die는 위아래가 막혀 열을 빼낼 길이 없다. 온도가 오르면 누설이 늘고(양의 피드백), 누설이 늘면 retention이 나빠지고, retention이 나빠지면 refresh를 더 해야 하고, refresh가 또 열을 낸다. HBM에서 전력 관리는 곧 **열 관리이자 성능 관리**다. 열 때문에 throttling(주파수 강하)이 걸리면 비싸게 쌓은 대역폭을 못 쓴다.

(3) 데이터센터 — 총량의 경제. AI 서버 한 대에 수백 GB~TB급 메모리가 들어가고, 그것이 수만 대 모인다. 셀 하나당 누설이 fJ 단위라도 곱하기 수조 개가 되면 메가와트가 된다. refresh 전력 1%를 줄이는 일이 전력비·냉각비에서 의미 있는 숫자가 된다.

설계자가 다루는 전력의 종류

면접에서 "전력"을 물으면 다음 분류로 구조화하면 좋다.

분류	정의	줄이는 주된 레버
Active(동적) 전력	read/write 등 실제 동작 중 스위칭으로 쓰는 전력	clock gating, DVFS, 활성 bank 최소화
Standby(누설) 전력	동작 안 해도 흐르는 leakage	multi-Vt, power gating, 저전압, deep power-down
Refresh 전력	데이터 유지를 위한 주기적 read-restore	self-refresh, 온도보상, PASR, fine-granularity refresh
I/O 전력	외부와의 고속 신호 토글·termination	저전압 인터페이스, 종단 최적화, 활성 시간 최소화

핵심 통찰: 로직에서는 active 전력이 보통 주연이지만, 메모리에서는 standby와 refresh가 — 즉 "아무것도 안 할 때 새는 전력"이 — 종종 주연이다. 이것이 메모리 저전력 설계가 로직과 결정적으로 다른 지점이다.

2. 전력 성분의 1st-principles — CV^2f 는 왜 그런 모양인가

CMOS 회로의 전력은 세 성분으로 분해된다. 식을 외우는 게 아니라 **왜 그런 모양인지**를 끝까지 따라가자.

2.1 동적(switching) 전력 — $P = \alpha \cdot C \cdot V^2 \cdot f$

게이트 출력이 0→1로 바뀔 때, 그 노드에 매달린 부하 커패시터 C를 전압 V까지 충전한다. 충전에 드는 에너지는 $\frac{1}{2}CV^2$ 이고, 다음에 1→0으로 방전할 때 그 $\frac{1}{2}CV^2$ 가 버려진다. 한 번의 완전한 토글(0→1→0)마다 커패시터에는 CV^2 의 에너지가 소모된다.

이제 시간당 전력으로 바꾸자.

- 1초에 클럭이 f번 된다.
- 그러나 모든 노드가 매 클럭 토글하는 건 아니다. 평균적으로 노드가 토글하는 비율이 **activity factor α** (0~1)다. 클럭 신호는 $\alpha \approx 1$ (또는 정의에 따라 0.5)에 가깝고, 데이터 신호는 보통 훨씬 작다.

→ $P_{dynamic} = \alpha \cdot C \cdot V^2 \cdot f$.

이 식에서 각 항을 줄이는 게 곧 저전력 레버다. 두 가지를 반드시 직관으로 잡아두자.

왜 주파수 f 에 비례하나? 토글 1회마다 CV^2 를 쓰는데, 1초에 토글 횟수가 곧 $f(\propto \alpha)$ 이기 때문이다. 빠를수록 같은 일을 더 자주 하니 전력이 비례해서 늘다. **선형**이다.

왜 전압 V 는 제곱인가? 충전 에너지 자체가 $\frac{1}{2}CV^2$ 로 V 의 제곱이기 때문이다. 직관적으로, 전압을 두 배 올리면 (a) 같은 커패시터를 두 배 높은 전압까지 채워야 하고, (b) 그 두 배 전압을 채우려면 두 배의 전하를 밀어 넣어야 한다 — 전압 $\times$ 전하 = 에너지가 $2\times 2 = 4$ 배가 된다. **이 제곱이 저전력 설계의 황금률**이다: 전압을 조금만 낮춰도 전력이 크게 떨어진다. DVFS·voltage island가 가장 강력한 무기인 이유가 바로 이 제곱에 있다.

2.2 단락(short-circuit) 전력

CMOS 인버터는 PMOS(pull-up)와 NMOS(pull-down)가 직렬로 $VDD \sim GND$ 사이에 놓인다. 정상 상태에서는 둘 중 하나만 켜져 직류 경로가 없다 — 이것이 CMOS의 위대함이다. 그러나 입력이 $0 \rightarrow 1$ 로 **전이하는 짧은 순간**, 입력 전압이 NMOS의 V_{th} 보다 높고 동시에 PMOS가 아직 완전히 안 꺼진 구간이 존재한다. 이때 PMOS와 NMOS가 **동시에 살짝 켜져** VDD 에서 GND 로 직접 관통 전류가 흐른다. 이것이 short-circuit power다.

핵심: short-circuit 전력은 **입력 천이 시간(slew)이 느릴수록 커진다**. 천이가 느리면 "둘 다 켜진" 구간이 오래 유지되기 때문이다. 그래서 buffer를 적절히 넣어 slew를 빠르게 유지하는 것이 short-circuit 전력 관리의 핵심이다. 보통 전체 동적 전력의 작은 비중(수~십수 %)이지만, slew가 망가진 노드에서는 무시 못 한다.

2.3 누설(leakage) 전력 — 미세화의 주인공

토글이 전혀 없어도, 트랜지스터가 "꺼져 있어도", 전류는 샌다. 이것이 $P_{leakage} = V \cdot I_{leak}$. 누설은 여러 경로의 합이다(질문은행 B7과 연결).

- **Subthreshold leakage(지배적)**. MOSFET은 V_{gs} 가 V_{th} 보다 낮아도 전류가 정확히 0이 아니다. 게이트 전압에 따라 전류가 **지수적으로** 변하며, off 상태에서도 미세한 누설이 흐른다. 결정적으로, 미세화로 성능을 위해 V_{th} 를 낮추면 off 상태의 누설이 **지수적으로 폭증**한다. 또 subthreshold 누설은 **온도에 지수적으로 민감**해서, 고온에서 급증한다 — 이것이 "고온일수록 leakage $\uparrow \rightarrow$ retention $\downarrow$ " 사슬의 물리적 뿌리다.
- **Gate(tunneling) leakage**. 게이트 산화막이 수 나노미터로 얇아지면, 전자가 양자역학적 터널링으로 산화막을 뚫고 샌다. high-k 유전체로 물리적 두께를 키워 완화한다.
- **Junction leakage / GIDL**. 역바이어스 PN 접합의 누설과, drain 근처 강한 전계가 만드는 GIDL(Gate-Induced Drain Leakage). **GIDL은 DRAM access 트랜지스터에서 특히 치명적**이다 — 셀 커패시터의 전하를 직접 빼내 retention을 망치기 때문이다.

왜 미세화로 leakage 비중이 커지나? 두 가지가 겹친다. (1) 트랜지스터 개수가 천문학적으로 늘어 누설의 총합이 커지고, (2) 성능을 위해 V_{th} 를 낮추고 oxide를 얇게 하면 개별 누설 자체가 커진다. 선단 공정에서는 idle 전력의 주범이 동적 전력이 아니라 **누설**인 경우가 흔하다. 그래서 "전원을 아예 끄는" power gating이 미세 공정에서 필수가 된다.

2.4 각 성분을 줄이는 레버 — 한눈에

성분	식/지배 변수	주된 레버	직관
동적	$\alpha \cdot C \cdot V^2 \cdot f$	DVFS($V \downarrow f \downarrow$), clock gating($\alpha \cdot f \downarrow$), 배치 최적화($C \downarrow$)	V 공급이 최강
단락	$slew \cdot V$	buffer로 slew 빠르게	"둘 다 켜진" 구간 단축
누설	$V \cdot I_{leak}(V_{th} \cdot T \cdot oxide)$	multi-Vt, power gating, body bias, 저전압	off 상태 자체를 더 꺼라

3. 동적전력 절감 — clock gating을 중심으로 (질문은행 D2, F_lowpower Q2/Q7)

동적전력 $P = \alpha \cdot C \cdot V^2 \cdot f$ 를 줄이는 네 갈래를 효과 순서로 본다: $V(\text{제공}) > \alpha \cdot f > C > (\text{그 외})$.

3.1 Clock gating — 가장 보편적이고 거의 공짜인 기법

무엇을. 어떤 레지스터 그룹이 이번 사이클에 새 값을 받을 필요가 없다면, 그 그룹으로 가는 클럭 자체를 차단한다. 그러면 (a) 클럭이 그 FF들까지 전파되지 않으니 클럭 트리의 토글이 사라지고, (b) FF 내부의 마스터-슬레이브 래치가 토글하지 않으니 FF 내부 switching도 사라진다. 클럭은 $\alpha \approx 1$ 인 가장 바쁜 신호이므로, 미사용 영역의 클럭을 끄는 것은 동적전력 절감의 가장 큰 단일 레버다. 합성 도구가 RTL의 enable 조건(`if(en) q<=d;`)을 보고 자동 삽입하기도 한다.

왜 단순 AND 게이트로 하면 안 되나 — glitch 문제. 직관적으로는 `gated_clk = clk AND enable` 이면 될 것 같다. 그러나 enable 신호가 클럭이 high인 동안에 바뀌면 어떻게 되나? clk이 1인데 enable이 1→0으로 떨어지면, gated_clk이 1에서 0으로 툭 떨어진다 — 이것은 의도하지 않은 클럭의 falling edge(=glitch 펄스) 다. 다운스트림 FF가 이 가짜 에지에 데이터를 잘못 래치할 수 있다. 반대로 enable이 high 구간에 0→1로 올라가면 가짜 rising edge가 생긴다. 클럭 신호에 생긴 glitch는 곧 잘못된 데이터 캡처이고, 메모리 컨트롤러에서는 명령 한 줄이 통째로 오동작하는 결과로 이어진다.

ICG 셀 — 왜 latch 기반인가. 해법은 enable을 클럭과 정렬하는 것이다. ICG(Integrated Clock Gating) 셀은 클럭이 low인 동안에만 enable 값을 통과시켜 latch에 가둔다(level-sensitive latch). 그 잠긴 enable을 clk과 AND한다. 결과적으로 enable은 클럭이 low일 때만 변할 수 있게 만들어지므로, 클럭이 high인 동안에는 gated_clk이 절대 흔들리지 않는다 → glitch-free. "latch로 enable을 한 박자 잡아 클럭에 정렬한다"가 핵심이다.

왜 enable 타이밍이 STA/CDC의 대상인가 — 지원자 직결. 여기서 지원자의 STA·CDC 경험이 정확히 닿는다.

- **STA 관점.** ICG의 latch는 클럭의 low phase(보통 falling edge 기준)에서 enable을 캡처한다. 따라서 enable 신호는 그 latch의 setup/hold 창 안에 안정해야 한다. enable 생성 로직이 너무 느리면 enable이 제때 안정되지 못해 setup 위반이 나고, gating이 의도대로 안 걸리거나 glitch가 생긴다. 즉 enable path는 별도의 timing path로 STA가 반드시 봐야 한다. 또 gated clock은 원래 클럭보다 트리 한 단계가 더 깊어지므로 clock skew·균형도 다시 봐야 한다. clock gating을 넣었더니 timing이 깨지는 것은 흔한 일이고, 그것을 잡는 게 STA closure다.
- **CDC 관점.** enable 신호가 다른 클럭 도메인에서 생성되면, 그 enable은 gating될 클럭 도메인으로 넘어오는 순간 CDC path가 된다. 동기화 안 된 enable이 metastable하게 흔들리면 gated clock에 glitch가 생긴다 — CDC 위반이 클럭 무결성 사고로 직결되는 대표 사례다. 메모리 Peri는 코어·I/O·DLL/PLL·refresh 등 다중 클럭 도메인이고, 각 bank를 clock gating하는 enable이 도메인 경계를 넘는 일이 흔하므로, bank별 clock gating의 enable CDC 검증은 그대로 지원자의 일이다.

이것이 D2 꼬리질문 "clock gating이 STA·CDC에 주는 영향?"의 핵심 답이고, F_lowpower Q2의 "gating으로 인한 timing 영향"의 정확한 그림이다.

coarse vs fine-grain. 큰 블록 하나에 enable을 거는 coarse gating은 절감폭이 크지만 enable 조건을 만들기 쉽고, 작은 FF 그룹마다 거는 fine-grain gating은 절감이 정밀하지만 enable 로직 오버헤드와 검증 부담이 커진다.

3.2 DVFS — 공급의 황금률을 직접 쓰는 기법

워크로드가 가벼울 때 전압 V 와 주파수 f 를 **함께** 낮춘다. 동적전력은 V^2f 에 비례하므로 절감폭이 크다(예: $V \cdot f$ 를 각각 0.8배로 내리면 동적전력은 $0.8^2 \times 0.8 \approx 0.51$ 배).

왜 V 를 낮추면 f 도 낮춰야 하나? 전압이 낮으면 트랜지스터가 약하게 켜져 게이트 지연이 늘어난다. 같은 주파수를 유지하면 늘어난 지연이 클럭 주기를 넘겨 **setup 위반**이 난다. 그래서 V 를 내리면 그만큼 주기를 늘려(f 를 낮춰) 타이밍을 지켜야 한다. V 와 f 는 한 쌍으로 움직인다. PMU(Power Management Unit)가 런타임에 부하를 보고 동작점(operating point)을 바꾼다. 메모리에서도 동작 속도 등급(speed bin)에 따라 전압을 바꾸는 것이 같은 원리다.

3.3 Voltage/frequency scaling, voltage island

칩을 여러 **전압 도메인(island)**으로 나눠, 성능이 덜 중요한 영역은 상시 낮은 전압으로 돌린다. 도메인 경계에는 **level shifter**(전압 변환)가 필요하다. 낮은 전압의 신호가 높은 전압 도메인의 게이트를 제대로 못 켜거나 누설을 일으키는 문제를 막기 위해서다(F_{lowpower} Q6). DVFS가 시간축의 전압 조절이라면, voltage island는 공간축의 전압 분할이다.

3.4 Operand isolation(데이터 게이팅)

clock gating이 "클럭을 끄는" 것이라면, operand isolation은 "**데이터 입력을 고정**"하는 것이다. 곱셈기 같은 큰 조합 로직의 결과가 이번 사이클에 쓰이지 않는다면, 그 입력을 이전 값으로 붙들어 두어(latch/AND로 hold) 조합 로직 내부가 불필요하게 토글하지 않게 한다. enable이 0일 때 입력이 변해도 내부 글리치 전파가 없어 동적전력을 아낀다. clock gating이 순차 로직(FF)을 겨냥한다면 operand isolation은 그 앞단 **조합 로직의 switching**을 겨냥한다. glitch power(불필요한 조합 글리치가 만드는 토글)를 줄이는 path balancing도 같은 가족이다.

4. 누설전력 절감 — multi-Vt와 power gating (질문은행 D1, F_{lowpower} Q3/Q4)

누설은 "off 상태 자체를 어떻게 더 끄느냐"의 싸움이다. 두 주력 무기는 셀 선택(multi-Vt)과 전원 차단(power gating)이다.

4.1 Multi-Vt — timing ↔ leakage trade-off의 정수

같은 기능의 표준 셀을 **문턱전압(V_{th})**이 다른 여러 버전으로 제공한다.

- **Low-Vt(LVT):** V_{th} 가 낮아 빠르게 켜진다 → **빠름**. 그러나 off 상태에서도 게이트가 "덜 단단히 꺼져" subthreshold 누설이 **크다**.
- **High-Vt(HVT):** V_{th} 가 높아 켜는 데 더 큰 전압이 필요하다 → **느림**. 대신 off 상태가 단단해 누설이 **작다**.

핵심 trade-off: 빠름과 저누설은 양립하기 어렵다(같은 공정에서). 그래서 합성·STA 도구가 **path별로 셀을 골라** 쓴다:

- **Critical path(타이밍 빡빡한 경로):** LVT를 써서 속도를 확보한다. 누설은 손해 보지만 어쩔 수 없다.

- **Non-critical path(타이밍 여유 있는 경로):** HVT로 바꿔 누설을 줄인다. 어차피 느려도 주기 안에 들어오니 속도 손해가 없다.

이것이 면접 단골 "leakage 줄이는 법"의 1순위 답이다. **요지: 타이밍 slack이 남는 곳마다 셀을 더 느리고 덜 새는 HVT로 swap한다 — 공짜로 누설을 줄이는 셈.** 도구가 timing closure 후 slack을 보며 자동으로 VT swap을 수행한다(이것이 STA와 직결되는 지점이다: slack 정보가 곧 누설 최적화의 입력이다).

4.2 Power gating — 누설을 "0에 가깝게" 만드는 기법

multi-Vt가 누설을 줄인다면, power gating은 누설을 아예 없앤다. idle 블록과 전원 사이에 **sleep transistor**를 넣어 전원을 끊는다.

- **Header(PMOS):** 블록과 VDD 사이에 PMOS를 넣어, sleep 시 VDD 공급을 차단(virtual VDD가 떠버림).
- **Footer(NMOS):** 블록과 GND 사이에 NMOS를 넣어, sleep 시 GND 경로를 차단.
- 이 sleep TR을 보통 **고-Vt**로 만든다(MTCMOS, Multi-Threshold CMOS) — 자기 자신의 누설을 최소화하려고.

전원을 끊으면 그 블록의 누설 경로 자체가 사라지므로 standby 누설이 거의 0이 된다. 그러나 **공짜가 아니다.** clock gating이 동적전력만 줄이고 전원은 켜둔다(누설은 남음)는 점과 결정적으로 다르다 — power gating은 누설까지 없애지만 그 대가로 다음 네 가지 장치가 반드시 필요하다(질문은행 D1 꼬리질문 "power gating의 부작용", F_lowpower Q3).

(1) Retention FF — state 보존. 전원을 끊으면 그 블록의 FF가 들고 있던 상태가 전부 날아간다. 깨어났을 때 처음부터 다시 시작할 수 없는 상태(예: 컨트롤러의 FSM, 설정 레지스터)는 **retention flop**에 보관한다. retention FF는 메인 전원이 꺼져도 살아 있는 **always-on supply**에 **연결된 shadow latch(balloon latch)**를 따로 가진다. sleep 직전에 메인 래치 값을 shadow로 복사(save)하고, wake 시 shadow에서 메인으로 복원(restore)한다. **always-on shadow latch에 state를 박제한다**가 핵심 그림이다.

(2) Isolation cell — 격리. 전원이 꺼진 도메인의 출력은 떠버린다(floating, 전기적으로 X/부정값). 이 떠 있는 값이 **여전히 켜져 있는 옆 도메인**으로 들어가면, 옆 도메인이 X를 받아 오동작하거나 PMOS/NMOS가 동시에 켜져 관통 전류가 흐른다. 그래서 꺼지는 도메인의 **출력단에 isolation cell**을 두어, sleep 동안 출력을 깨끗한 0 또는 1로 **clamp**한다. "꺼진 방의 문틈을 막아 그 방의 어둠이 켜진 방으로 새지 않게 한다"는 그림이다. isolation enable 신호도 power state에 맞춰 제어돼야 하므로 이것 역시 검증 대상이다.

(3) Rush current(inrush) — 켜는 순간의 돌입 전류. 전원을 끊었다가 다시 켤 때, virtual VDD 노드와 그에 매달린 모든 decap이 한꺼번에 충전되며 **큰 돌입 전류**가 순간적으로 흐른다. 이 rush current가 너무 크면 전원망에 IR drop을 일으켜 **옆에서 멀쩡히 돌던 다른 블록까지 전압이 출렁여 오동작**할 수 있다. 그래서 sleep TR을 한꺼번에 켜지 않고, 작은 TR부터 **단계적으로 켜는 power-on 시퀀싱**(weak switch 먼저 → strong switch)으로 rush current를 완만하게 만든다.

(4) 전원 시퀀싱과 wake-up latency. save → power-off → (idle) → power-on(rush 제어) → restore → isolation 해제 → 동작 재개라는 **순서가 정확히 지켜져야** 한다. 순서가 틀리면 — 예컨대 isolation을 너무 일찍 풀면 — 아직 안 켜진 도메인의 X가 새어 나간다. 그리고 이 전체 과정에 시간이 걸리므로(**wake-up latency**), power gating은 "충분히 오래 idle할 블록"에만 이득이다. 자주 켜고 끄면 save/restore-rush 오버헤드가 절감보다 커진다.

4.3 Body/back bias — 문턱전압을 동적으로 조절

트랜지스터의 body(기판/well)에 전압을 걸어 **Vth 자체를 실시간으로 바꾼다**(body effect). idle 시 **reverse body bias(RBB)** 를 걸어 Vth를 높이면 누설이 준다(느려지지만 idle이니 상관없다). 성능이 필요할 때는 **forward body bias(FBB)** 로 Vth를 낮춰 속도를 회복한다. multi-Vt가 설계 시점에 셀을 고정 선택하는 정적 기법이라면, body bias는 런타임에 Vth를 미는 동적 기법이다. 공정·웰 구조 지원이 필요해 모든 공정에서 쓰진 않는다.

4.4 Stack effect — 직렬로 쌓으면 덜 샌다

off 트랜지스터를 두 개 직렬로 쌓으면 하나일 때보다 누설이 크게 준다(stacking effect). 직관: 두 TR이 직렬이면 가운데 노드의 전압이 자기바이어스(self-bias)되어, 위쪽 TR의 Vgs가 음수 쪽으로 밀리고 Vds도 줄어 subthreshold 누설이 지수적으로 감소한다. 그래서 NAND 게이트 같은 직렬 구조는 입력 조합에 따라 누설이 달라지고, 합성 도구는 누설을 줄이려고 일부러 stack을 활용하기도 한다. multi-input 게이트의 "어떤 입력 패턴이 누설을 최소화하나"를 고르는 input vector control도 같은 원리다.

4.5 전력 의도를 코드로 — UPF/CPF (F_lowpower Q5)

power gating·isolation·retention·level shifter·power state는 RTL의 기능 기술과 별개의 "전력 구조" 다. 이것을 RTL에 직접 섞어 짜면 기능과 전력이 뒤엉켜 재사용·검증이 망가진다. 그래서 **전력 의도(power intent)** 를 별도 파일로 적는 표준이 **UPF(IEEE 1801, Unified Power Format)** — Cadence 진영의 CPF도 같은 목적이다.

UPF는 무엇을 적나: power domain(어디서 어디까지가 한 전원 영역인가), supply net(어떤 전원이 공급되나), isolation/retention/level-shifter 전략(어디에 어떤 셀을 끼우나), **power state table**(어떤 도메인이 켜지고 꺼지는 조합이 유효한가)을 기술한다. 이점:

- **같은 RTL, 다른 전력 구조.** 기능을 안 건드리고 전력 구조만 바꿔 실험할 수 있다.
- **도구 일관성.** 합성·검증·배치 도구가 같은 UPF를 읽어, "isolation 누락" "retention 빠짐" "켜진 도메인이 꺼진 도메인 신호를 격리 없이 받음" 같은 실수를 **정적으로 잡는다**(power-aware 검증).
- **검증 대상화.** 이 정적 검사는 지원자의 CDC sign-off와 사고방식이 같다 — 구조적 규칙을 정의하고 위반을 자동으로 찾아내는 일이다. power state table은 본질적으로 "어떤 도메인 조합이 합법인가"를 명세하는 것이고, false violation을 걸러내는 작업은 CDC/RDC에서 하던 그것이다.

5. 메모리 특유의 저전력 — retention·refresh·standby (질문은행 D3, A2)

여기서부터가 메모리 설계자만의 영토다. 로직에는 없는, "데이터를 살아 있게 유지하는 전력"의 세계다.

5.1 왜 refresh인가 — 누설이 retention을 정한다

DRAM 셀은 1T1C — 트랜지스터 하나, 커패시터 하나. 데이터는 커패시터에 전하로 저장된다(전하 있음=1, 없음=0, 또는 그 반대). 그런데 **커패시터의 전하는 access 트랜지스터의 누설·접합 누설·GIDL 등으로 시간이 지나면 샌다.** 전하가 sense amp가 구분할 수 있는 한계 아래로 떨어지기 전에, 셀을 한 번 읽어 다시 가득 채워줘야 (restore) 한다. 이것이 **refresh**다.

- **retention time:** 한 셀이 데이터를 유지하는 시간. 통상 64ms 급(변동·공정/온도 의존). 이 시간 안에 **모든 row**를 한 번씩 refresh해야 한다.

- 누설↑ → retention↓ → refresh 주기↓(더 자주) → refresh 전력↓. 1장에서 본 사슬이 여기서 물리로 완성된다. 미세화로 셀 커패시턴스가 작아지고 누설이 커질수록 retention이 나빠져 refresh 부담이 커진다.

refresh에는 두 종류가 있다(A2/D3 꼬리질문).

- **Auto-refresh:** 정상 동작 중, 컨트롤러가 주기적으로 refresh 명령을 보내고 DRAM 내부 카운터가 row를 순회한다. 동작과 병행하므로 그 시간만큼 access를 못 한다(성능 오버헤드).
- **Self-refresh:** 컨트롤러가 손을 떼고 DRAM이 스스로 내부 타이머·카운터로 refresh를 돌리는 **저전력 standby 모드**. 외부 클럭·인터페이스를 끄고 최소한의 회로만 살려 데이터만 유지한다. 모바일·idle 상태의 핵심 저전력 모드다.

5.2 self-refresh에서 전력을 더 줄이는 법

D3 꼬리질문 "self-refresh에서 전력을 더 줄이는 법?"이 면접 포인트다. 여러 레버가 있다.

(1) **온도보상 refresh — TCSR / 온도 종속 주기.** retention은 온도에 강하게 의존한다(누설이 온도에 지수적이므로). 저온에서는 전하가 천천히 새므로 refresh를 덜 해도 된다. 그런데 보수적으로 항상 고온 기준(가장 짧은 retention)으로 refresh하면, 저온일 때 불필요하게 자주 refresh해 전력을 낭비한다. **TCSR(Temperature-Compensated Self-Refresh)**은 내부 온도 센서로 칩 온도를 읽어, **온도가 낮으면 refresh 주기를 늘려** refresh 횟수를 줄인다. "필요한 만큼만 refresh한다"는 적응형 제어다. 이 온도→주기 매핑 로직이 곧 디지털 제어 로직이고 검증 대상이다.

(2) **PASR — Partial Array Self-Refresh.** standby 동안 메모리 전체가 다 살아 있을 필요가 없을 때가 많다(예: OS가 일부 영역만 유효 데이터를 둬). **PASR은 self-refresh 대상을 array의 일부(예: 절반, 1/4 bank)로 제한한다.** refresh하지 않는 영역의 데이터는 버려지지만(그 영역을 안 쓰기로 OS·컨트롤러가 합의), 그만큼 refresh 횟수가 줄어 전력이 준다. "유지할 데이터만 유지한다"는 발상이다.

(3) **Fine-granularity / per-bank refresh.** refresh를 bank 단위로 잘게 쪼개면, 한 bank를 refresh하는 동안 다른 bank는 access할 수 있어 성능·전력 분산에 유리하다.

(4) **Weak cell 관리.** 셀마다 retention이 다르다 — 유독 빨리 새는 **weak cell**이 분포의 꼬리에 있다. refresh 주기는 보수적으로 "가장 약한 셀"에 맞춰지므로, weak cell 몇 개 때문에 전체가 자주 refresh된다. weak cell을 redundancy로 구제하거나 target refresh로 따로 챙기면, 나머지의 refresh 주기를 늘려 전력을 아낄 수 있다.

5.3 LPDDR deep power-down

self-refresh보다 더 깊은 모드. **deep power-down**은 데이터 유지조차 포기하고(또는 최소 회로만 남기고) 거의 모든 전원을 끊어 누설을 극단적으로 줄인다. 다시 쓰려면 재초기화가 필요하다. "잠깐 졸기(self-refresh)"와 "전원을 거의 끄기(deep power-down)" 사이의 다단계 저전력 모드가 LPDDR의 특징이다. 4장의 power gating 개념이 메모리 전체 규모로 적용된 것으로 이해하면 된다.

이 모든 모드 전환 — 언제 self-refresh로 들어가고, 온도에 따라 주기를 어떻게 바꾸고, 어떤 array를 PASR로 끄고, deep power-down에서 어떻게 깨어나는가 — 는 전부 **유한상태기계(FSM)와 타이머로 구현되는 디지털 제어 로직**이며, 모드 경계는 종종 클럭 도메인 경계(self-refresh에서 내부 저속 클럭으로 전환)라 **CDC가 끼고**, 타이밍이 빡빡해 **STA가 본다**. 메모리 디바이스의 물리를 몰라도, 이 제어 로직의 검증은 지원자가 하던 일 그대로다.

6. 신뢰성 1 — DRAM RowHammer (질문은행 D6)

이제 누설이 신뢰성으로 번지는 첫 갈래. RowHammer는 면접에서 점점 중요해지는 주제다(보안과 얽혀서).

6.1 메커니즘 — 왜 인접 row가 흔들리나

DRAM array는 word line(WL, 가로)과 bit line(BL, 세로)의 격자다. 한 row를 access하려면 그 row의 WL을 **activate**(전압을 올려 access TR을 줌) 하고, 동작 후 **precharge**한다. WL이 오르내리는 것이다.

특정 row를 **매우 빠르게, 반복적으로** activate/precharge하면(이 row를 "aggressor"라 한다), 그 WL의 반복적인 전압 토글이 **물리적으로 바로 옆 row("victim")의 셀에 영향을 준다**. 메커니즘은 복잡하지만 핵심은:

- aggressor WL의 전계가 인접 셀의 access TR에 결합되어 **전하 누설을 가속한다**(특히 앞서 본 GIDL 등이 강화된다).
- 셀 간 간격이 좁아진 미세 공정일수록 이 결합(coupling)이 강해진다 — **미세화가 RowHammer를 악화시킨다**.

결과: victim row의 셀 전하가 정상 retention 시간보다 **훨씬 빨리** 한계 아래로 떨어진다. 즉, 다음 refresh가 오기 전에 전하를 잃어 **bit이 뒤집힌다(bit flip)**. "특정 행을 망치처럼 두드리면(hammer) 옆 행의 데이터가 깨진다"는 그림이다.

6.2 왜 보안 이슈인가

RowHammer가 무서운 건 단순 신뢰성 오류를 넘어 **공격에 악용되기** 때문이다. 공격자가 자기 권한 안의 메모리를 의도적으로 반복 access(hammering)해서, **자기가 건드릴 수 없는 인접 영역(예: 페이지 테이블, 권한 비트)의 bit을 뒤집어** 권한 상승을 노릴 수 있다. 물리적 결합을 소프트웨어로 트리거하는 보안 취약점이라 매우 위험하다.

6.3 대응 — TRR과 추가 refresh, DDR5 강화

근본 대응은 "victim이 깨지기 전에 미리 refresh해주는 것"이다.

- **TRR(Target Row Refresh)**. 특정 row가 비정상적으로 자주 activate되는지를 **모니터링**하다가, 임계치를 넘으면 그 row의 **인접 row들을 추가로 refresh**한다. aggressor가 victim을 깨기 전에 victim을 다시 채워 막는다. 모니터링 방식은 카운터 기반(activation count) 등 여러 구현이 있다.
- **DDR5의 강화**. DDR5에서는 RowHammer 대응이 더 정교해졌다. **RFM(Refresh Management)** 같은 메커니즘으로, 컨트롤러가 activation 빈도를 추적해 일정량이 쌓이면 DRAM에 management 명령을 보내 추가 refresh 기회를 준다. on-die에서 aggressor를 추적하는 회로(예: 확률적·counter 기반 추적)도 강화됐다. mitigation의 디테일은 세대·벤더별로 발전 중이다.

6.4 지원자 연결

TRR/RFM은 본질적으로 "**access 패턴을 모니터링 → 임계 판단 → 추가 refresh 스케줄**" 하는 디지털 제어 로직이다. activation 카운트를 세고, 임계와 비교하고, refresh 명령을 끼워 넣는 일 — 카운터·비교기·스케줄러·FSM이다. 그리고 이 로직은 정상 refresh·access와 타이밍이 얽혀 **STA로 검증**되고, 컨트롤러-DRAM 명령 인터페이스에서 **프로토콜 정합성(SVA/formal로 검증)**의 대상이다. 즉 RowHammer 대응은 디바이스 물리 위에 얹힌 **검증 가능한 제어 로직**이고, 그 검증이 지원자의 일이다.

7. 신뢰성 2 — NAND read disturb·program disturb·P/E wear (질문은행 D4, D5)

NAND의 신뢰성 문제는 RowHammer와 **놀랍도록 대칭적인 구조**를 갖는다. "반복 동작 → 인접/자기 셀 교란 → 모니터·완화"라는 패턴이 똑같다. 이 대칭을 면접에서 짚으면 통찰력이 드러난다.

7.1 NAND 셀과 교란의 무대

NAND 셀은 floating gate(또는 charge trap)에 전하를 가둬 V_{th} 를 바꿔 데이터를 저장한다(전하량 = V_{th} = 데이터). 셀들이 **NAND string**으로 직렬 연결되어, 한 셀을 읽으려면 같은 string의 **다른(비선택) 셀들은 무조건 켜둬야**(pass voltage를 걸어야) 한다. 여기서 교란이 시작된다.

7.2 Read disturb — 읽기가 옆 셀을 흔든다

특정 페이지를 **read**하면, 같은 string의 비선택 셀들에 **pass voltage(V_{pass})**가 걸린다. 이 pass voltage는 약하지만, 비선택 셀의 floating gate로 전자를 **미세하게 주입**하는 방향으로 작용한다. read를 **수만~수십만 번 반복**하면, 이 미세 주입이 누적되어 비선택 셀의 V_{th} 가 조금씩 올라가고, 결국 데이터가 잘못 읽히는 **bit error**가 난다. "읽기만 했는데 옆 셀이 망가진다"는 점에서 RowHammer와 정확히 대칭이다.

대응: read 횟수를 **모니터링**하다가 임계를 넘으면 그 블록을 **다른 곳으로 재기록(read reclaim/refresh)** 해 누적된 disturb를 리셋하고, 평소엔 **ECC**로 소수 bit error를 정정한다.

7.3 Program disturb — 쓰기가 옆 셀을 흔든다

한 셀을 **program**(전자 주입)할 때, 같은 워드라인·비트라인을 공유하는 **이웃 셀에 의도치 않은 전계**가 걸려 약하게 프로그램되는 현상. read disturb가 "읽기 누적"이라면 program disturb는 "쓰기 시 이웃 결함"이다. 둘 다 **비선택 셀의 의도치 않은 V_{th} 변화**라는 점에서 같은 가족이다.

7.4 P/E wear — 반복이 산화막을 늙힌다

NAND의 program/erase는 전자를 tunnel oxide를 통과시켜 floating gate에 넣거나 뺀다(터널링). 이 **터널링이 매번 tunnel oxide를 조금씩 손상**시킨다 — 산화막에 trap이 생기고 절연이 열화된다. 반복할수록(P/E cycle이 쌓일수록) 산화막이 늙어 전하 보존력이 떨어지고, 결국 셀이 데이터를 못 잡는다. 이것이 NAND의 **수명(P/E cycles 제한)**이다. SLC > MLC > TLC > QLC 순으로, 한 셀에 더 많은 bit(더 촘촘한 V_{th} 레벨)를 옥여넣을수록 마진이 작아져 같은 열화에도 더 빨리 한계에 닿는다 → 수명이 짧다.

대응: **wear-leveling**(컨트롤러가 쓰기를 전체 블록에 골고루 분산해 특정 블록만 빨리 늙는 걸 방지), bad block 관리, 강력한 ECC, read reclaim — 모두 컨트롤러 펌웨어·로직의 일이다.

7.5 RowHammer ↔ NAND disturb — "반복-교란-완화"의 대칭

축	DRAM RowHammer	NAND read/program disturb	NAND P/E wear
반복하는 동작	인접 row activate	read(또는 program)	program/erase
교란받는 대상	인접 row 셀(전하 누설 가속)	비선택/이웃 셀(Vth 변화)	tunnel oxide(영구 열화)
가역성	가역(refresh로 복구)	가역(reclaim으로 복구)	비가역(수명 소진)
모니터	activation count	read count	P/E count
완화	TRR/RFM 추가 refresh	read reclaim·ECC	wear-leveling·ECC·bad block

핵심 한 문장: "반복 동작이 인접/자기 셀을 교란하고, 그것을 카운트로 모니터링해 미리 완화한다"는 구조가 DRAM과 NAND에서 동형이다. 단 P/E wear만은 산화막의 영구 손상이라 비가역(수명)이라는 점이 다르다. 그리고 이 모니터·완화 로직은 전부 디지털 제어 + 검증 대상이다.

8. 신뢰성 3 — on-die ECC (질문은행 D8, C7)

8.1 왜 on-die ECC가 "필요해졌나"

D8의 핵심은 "왜 이제 와서 칩 안에 ECC를 넣게 됐나"다. 답은 1·2장의 사슬에 있다. 미세화 → 셀 작아짐·누설↑ → 셀 하나가 비트를 잘못 들 확률(single-bit error rate)↑. 예전엔 셀이 충분히 견고해 칩 자체가 에러 없이 동작했지만, 미세화가 진행되며 통계적으로 셀 어레이 안에서 single-bit error가 무시 못 할 빈도로 발생하게 됐다. 이 늘어난 raw bit error를 칩이 내부에서 스스로 정정하지 않으면 양품 수율과 신뢰성이 무너진다. 그래서 DDR5부터 on-die ECC가 표준에 포함됐다.

8.2 SECEDED 직관 — 어떻게 비트를 고치나

ECC의 기본형은 SECEDED(Single Error Correct, Double Error Detect). 직관:

- 데이터 비트들에 parity(검사) 비트 몇 개를 추가로 저장한다. 예컨대 64비트 데이터에 8비트 parity(총 72비트)를 붙인다.
- parity는 데이터 비트들의 특정 조합의 XOR로 계산된다. 읽을 때 다시 계산한 parity와 저장된 parity를 비교(syndrome)한다.
- syndrome이 0이면 에러 없음. 0이 아니면, 그 패턴이 "몇 번째 비트가 틀렸는지"를 정확히 가리키도록 parity를 설계했기 때문에(Hamming code), 그 비트를 뒤집어 고친다(SEC). 추가 parity 한 개를 더 두면 2비트 에러는 정정은 못 해도 검출(DED) 할 수 있다.
- 핵심 직관: "여분의 검사 비트로 데이터 공간에 거리(distance)를 만들어, 1비트가 튀어도 가장 가까운 올바른 코드워드로 되돌릴 수 있게 한다." 1비트 오류는 정정, 2비트 오류는 적어도 "틀렸다"고 알린다.

이 인코더(쓸 때 parity 생성)/디코더(읽을 때 syndrome 계산·정정)는 XOR 트리로 구현되는 순수 디지털 로직 — 곧 Peri의 검증 대상이다(C7과 연결).

8.3 on-die ECC vs system ECC — 역할이 다르다

면접에서 자주 묻는 구분(D8 꼬리질문).

	on-die ECC	system-level ECC
위치	DRAM 칩 내부	메모리 컨트롤러/CPU 쪽(DIMM 단위, 예: ECC DIMM)
막는 대상	칩 내부 셀 어레이의 single-bit error (미세화로 늘어난 것)	칩-시스템 경로 전체의 에러(다중 칩, 링크, chipkill 등)
가시성	호스트에 거의 투명(칩이 알아서 고침)	호스트가 인지·관리, 더 강한 코드(다중비트, 칩 단위 fail)
목적	양산 수율·기본 신뢰성 유지	시스템 차원의 가용성·서버 신뢰성

핵심: 둘은 경쟁이 아니라 계층적 보완이다. on-die ECC는 칩이 "출하 가능한 품질"을 유지하려고 내부에서 자잘한 에러를 닦아내고, system ECC는 칩 하나가 통째로 죽어도 시스템이 살아남게 하는 더 큰 그물이다. on-die ECC가 가린 에러가 system ECC의 통계에 안 잡혀 디버깅이 헛갈리는(서로의 정정이 겹치는) 미묘함도 있다.

8.4 HBM의 ECC

HBM은 워낙 광폭·고속·고밀도라 신뢰성이 핵심이라, 내부 ECC와 link ECC(전송 경로 보호) 등을 갖춘다. 적층 구조에서 한 die의 에러가 전체 스택의 신뢰성을 좌우하므로 ECC 비중이 더 크다.

9. 소자 aging — NBTI·HCI·TDDB·EM (질문은행 D7)

지금까지가 "셀 데이터의 교란"이었다면, aging은 트랜지스터·배선 자체가 시간이 지나며 늙는 문제다. 그리고 이것이 지원자의 STA·corner 사고와 가장 직접적으로 만난다.

9.1 네 가지 열화 메커니즘

- **NBTI(Negative Bias Temperature Instability)**. PMOS에 음의 게이트 바이어스가 고온에서 오래 걸리면 게이트 산화막 계면에 trap이 쌓여 V_{th} 가 점점 올라간다 → PMOS가 느려진다. 시간·온도·전압 의존. 회로가 늙으면서 점점 느려지는 대표 원인.
- **HCI(Hot Carrier Injection)**. 트랜지스터가 켜질 때 채널의 일부 전자가 강한 전계에 가속되어 "뜨거워진(hot)" 채로 산화막에 박힌다 → 역시 특성 열화(V_{th} shift, 이동도 저하). 스위칭이 잦은 노드에서 누적(toggle 의존).
- **TDDB(Time-Dependent Dielectric Breakdown)**. 게이트 산화막에 전계가 오래 걸리면 결함이 누적되다가 어느 순간 절연이 깨져 게이트 누설/단락이 된다. 얇은 oxide·고전압·고온일수록 빨리 온다. 시간 의존 고장.
- **EM(Electromigration)**. 금속 배선에 큰 전류 밀도가 오래 흐르면, 전자가 금속 원자를 물리적으로 밀어내 (momentum transfer) 한쪽엔 void(공극), 다른 쪽엔 hillock(돌출)이 생긴다 → 배선 저항 증가·단선. 전류 밀도·온도 의존.

9.2 왜 EM은 power/clock net에서 특히 critical인가

EM은 평균 전류(또는 단방향 전류)가 큰 배선에서 심하다. 신호선은 0↔1을 오가며 전류 방향이 바뀌어 평균이 상쇄되는 경향이 있지만:

- Power/ground 배선은 한 방향으로 큰 DC 전류가 지속적으로 흐른다 → EM에 가장 취약.
- Clock 배선은 칩에서 가장 자주·균일하게 토글하며($\alpha \approx 1$) 큰 buffer를 구동해 전류 밀도가 높다 → 역시 취약.

그래서 EM 검증은 power grid와 clock tree에 집중된다. 배선을 굵게(전류 밀도↓) 하거나 전류를 분산해 대비한다.

9.3 설계의 대비 — guard band / derating (STA corner 사고와 동일)

aging은 "지금엔 멀쩡한데 5년 뒤엔 느려질" 열화다. 설계는 미래의 열화를 미리 마진으로 흡수한다. 이것이 guard band(가드 밴드)/derating이다.

- NBTI/HCI로 회로가 수명 말기에 X% 느려질 것을 예측해, STA에서 그만큼 delay를 더 비관적으로(derate) 잡고 타이밍을 달는다. 즉 "신품 기준"이 아니라 "늙은 기준(end-of-life)"으로도 setup이 만족하도록 마진을 둔다.
- TDDb/EM은 수명(MTTF) 목표를 정해 전압·전류 밀도·온도 한계를 그 안에 들도록 설계한다.

여기가 지원자의 핵심 브릿지다. STA에서 PVT corner·OCV·derating으로 마진을 잡던 그 사고가 그대로 aging guard band다. corner를 보는 이유("최악 조건에서도 타이밍이 달해야 한다")와 aging derating을 보는 이유("늙은 최악 상태에서도 달해야 한다")는 동일한 마진 사고다. D7 꼬리질문 "aging이 timing margin에 주는 영향 (설계 시 derating)?"의 답이 바로 이것이다. 메모리 디바이스를 몰라도, "마진을 어떻게 잡는가"는 지원자가 PrimeTime으로 매일 하던 일이다.

10. 전부를 잇는 한 줄 — 왜 이 모든 것이 "지원자의 영역"인가

이 문서의 모든 갈래는 하나의 사슬로 수렴한다.

leakage → retention → refresh. 누설이 retention을 정하고, retention이 refresh 주기를 정한다. 열 → leakage → retention. 열이 누설을 키워 같은 사슬을 재촉한다(HBM에서 특히). 그리고 RowHammer 대응 (TRR/RFM), NAND disturb 완화(reclaim), on-die ECC(인코더/디코더), aging 대비(guard band), 저전력 모드 전환(self-refresh·PASR·TCSR·power gating)은 예외 없이 —

"제어 디지털 로직 + 타이밍 검증"으로 구현·검증된다.

- 카운터로 access를 세고(RowHammer/disturb), 온도로 주기를 바꾸고(TCSR), array를 부분만 켜고(PASR), 전원을 시퀀싱하고(power gating), parity로 비트를 고치는(ECC) — 전부 FSM·카운터·비교기·XOR 트리다.
- 그 로직의 enable이 도메인을 넘으면 CDC, 뽀빠한 타이밍은 STA, 클럭을 끄는 게이팅은 clock gating 검증, 프로토콜 준수는 SVA/formal, 미래 열화는 derating corner.

즉 메모리 소자의 물리(셀·커패시터·산화막)는 겉이지만, 그 물리를 "관리하는 로직"은 100% 지원자가 하던 일이다. 면접의 본질 명제는 "용어가 같다"가 아니라 "문제가 같다"이다 — 마진을 잡고, 도메인을 건너는 신호를

동기화하고, 클럭을 안전하게 끄고, 위반을 자동으로 걸러내는 그 문제.

11. 면접 연결 — 30초 / 1분 골격과 브릿지

각 질문에 대해 30초(핵심) → 1분(확장) → 브릿지 한 문장의 3단으로 준비한다.

Q. "Leakage를 줄이는 법은?" (D1)

- **30초:** "셀 선택과 전원 차단 둘로 봅니다. **multi-Vt**로 타이밍 여유가 있는 경로를 느리지만 덜 새는 high-Vt 셀로 swap하고, idle 블록은 **power gating**으로 전원을 끊어 누설 자체를 없앱니다. 거기에 body bias, stack effect, 저전압이 더해집니다."
- **1분 확장:** multi-Vt가 timing↔leakage trade-off라는 점(critical엔 LVT, slack 있는 곳엔 HVT — slack 정보가 곧 최적화 입력), power gating은 retention FF·isolation cell·rush current·시퀀싱이라는 부작용 패키지를 동반한다는 점, 메모리에선 이 누설이 standby/retention 전력으로 직결된다는 점까지.
- **브릿지:** "타이밍 slack을 보며 셀을 swap하는 건 제가 PrimeTime으로 timing closure 하며 slack을 다루던 일과 정확히 같습니다."

Q. "Clock gating의 부작용은?" (D2)

- **30초:** "enable이 클럭 high 중에 바뀌면 **glitch(가짜 클럭 에지)**가 생겨 오작동합니다. 그래서 ICG 셀이 클럭 low 동안 enable을 latch로 잡아 glitch-free로 만듭니다. 부작용은 **enable path의 setup 타이밍과 clock skew**, 그리고 enable이 다른 도메인에서 오면 **CDC** 문제입니다."
- **1분 확장:** glitch가 왜 데이터 손상인지, ICG latch가 어떻게 정렬하는지, gated clock이 트리를 깊게 해 skew를 다시 봐야 하는 점, 메모리 Peri의 bank별 gating에서 enable이 도메인을 넘는 점.
- **브릿지:** "enable의 setup 검증은 STA, 도메인 넘는 enable의 동기화는 CDC — 제 핵심 업무 그대로입니다. bank별 clock gating 검증이 곧 제가 하던 일입니다."

Q. "RowHammer와 TRR은?" (D6)

- **30초:** "특정 row를 반복 activate하면 그 WL의 전계가 **인접 row의 전하 누설을 가속**해 다음 refresh 전에 bit이 뒤집힙니다. 보안 공격에도 쓰입니다. **TRR**은 activation을 모니터링하다 임계를 넘으면 인접 row를 **추가 refresh**해 막고, DDR5는 RFM으로 강화했습니다."
- **1분 확장:** 왜 미세화가 악화시키는지(셀 간격↓→결합↑), 보안 관점(권한 비트 flip→권한 상승), DDR5 RFM·on-die 추적 회로.
- **브릿지:** "TRR은 access를 카운트해 임계를 보고 refresh를 끼우는 제어 로직 + 타이밍 검증입니다 — 제 영역입니다. NAND read disturb와도 '반복-교란-모니터-완화'로 같은 구조죠."

Q. "Self-refresh는?" (D3)

- **30초:** "컨트롤러가 손을 떼고 DRAM이 스스로 내부 타이머로 refresh를 돌리는 **저전력 standby 모드**입니다. 전력을 더 줄이려면 **온도보상(TCSR)**으로 저온일 때 주기를 늘리고, **PASR**로 유효 영역만 refresh하고, weak cell을 구제해 전체 주기를 늘립니다."
- **1분 확장:** retention이 누설-온도에 의존한다는 물리, auto vs self-refresh 차이, deep power-down까지의 다단계 모드.

- 브릿지: "모드 전환·온도→주기 매핑은 FSM·타이머 디지털 로직이고, self-refresh의 저속 내부 클럭 전환은 CDC가 겁니다 — 제가 검증하던 그대로입니다."

Q. "On-die ECC는 왜?" (D8)

- 30초: "미세화로 셀 어레이의 **single-bit error rate**가 올라가 칩이 그냥은 양품 신뢰성을 못 맞춥니다. 그래서 DDR5부터 칩 내부에 ECC를 넣어 자잘한 에러를 스스로 **SECEDED**로 정정합니다. system ECC가 막는 칩-시스템 레벨 에러와는 계층이 다른 보완 관계입니다."
- 1분 확장: SECEDED 직관(parity로 거리를 만들어 1비트는 정정·2비트는 검출), 인코더/디코더가 XOR 트리 디지털 로직, on-die vs system ECC 역할 표.
- 브릿지: "ECC 인코더/디코더는 Peri 디지털 로직이라 합성·정합성·프로토콜 검증 대상 — 제 LEC·SVA 경험이 닿습니다."

통합 브릿지 — "마진·검증 사고는 같다"

"제가 STA에서 PVT corner와 derating으로 마진을 잡던 사고는 그대로 **aging guard band**(NBTI/HCI로 늘어 느려질 만큼 미리 비관적으로 타이밍을 닫는 것)입니다. CDC에서 도메인 넘는 신호를 동기화하던 일은 **clock gating**의 enable CDC·self-refresh 클럭 전환으로 이어집니다. 즉 메모리 소자는 새로 배우지만, 그 소자를 관리하는 제어 로직의 검증은 제가 4년간 하던 일과 같은 문제입니다."

12. 스스로 점검 — 다음 질문에 막힘없이 답할 수 있는가

1. 동적전력 식 $P=\alpha \cdot C \cdot V^2 \cdot f$ 에서 **V가 왜 제곱**이고 **f가 왜 1차**인지, 충전 에너지 $\frac{1}{2}CV^2$ 부터 유도해 설명할 수 있는가?
2. 미세화로 누설 비중이 왜 커지는가를 V_{th} ·oxide·트랜지스터 수 세 가지로 나눠 말할 수 있는가? subthreshold가 온도에 지수적이라는 점이 retention과 어떻게 연결되는가?
3. **ICG 셀이 latch 기반인 이유**(glitch)를 enable이 클럭 high 중 바뀌는 시나리오로 그려 설명하고, 왜 그것이 STA(enable setup)와 CDC(도메인 넘는 enable)의 대상이 되는지 말할 수 있는가?
4. **multi-Vt의 timing↔leakage trade-off**를 critical/non-critical path와 slack 개념으로 설명하고, 도구가 어떻게 자동 swap하는지 말할 수 있는가?
5. **power gating의 부작용 4종**(retention FF, isolation cell, rush current, 시퀀싱)을 각각 "없으면 무슨 일이 나는가"로 설명할 수 있는가? clock gating과의 차이(동적 vs 누설)는?
6. **leakage → retention → refresh** 사슬을 물리(누설이 커패시터 전하를 뺏다)부터 한 호흡에 말하고, self-refresh에서 전력을 더 줄이는 법(TCSR/PASR/weak cell)을 딸 수 있는가?
7. **RowHammer 메커니즘**(반복 activate→인접 row 누설 가속→bit flip)과 보안 위험, TRR/RFM 대응을 설명하고, 그것이 디지털 제어 로직임을 말할 수 있는가?
8. **RowHammer ↔ NAND read disturb ↔ P/E wear**의 "반복-교란-모니터-완화" 대칭을, 특히 P/E wear만 비가역(수명)인 차이까지 표로 그릴 수 있는가?
9. **on-die ECC가 "왜 이제 필요해졌나"**(미세화→single-bit error↑)를 답하고, SECEDED 직관(parity로 거리를 만든다)과 system ECC와의 역할 차이를 구분할 수 있는가?
10. **EM이 왜 power/clock net에서 critical**한지(단방향/잦은 큰 전류)를 설명하고, NBTI/HCI/TDDDB와 함께 **aging guard band**가 STA corner/derating 사고와 같다는 브릿지를 한 문장으로 말할 수 있는가?

11. UPF가 전력 의도를 RTL과 분리해 기술하는 이유와, power state table의 정적 검증이 CDC sign-off 사고와 어떻게 닮았는지 설명할 수 있는가?
12. (종합) "메모리 소자는 갭이지만 그 소자를 관리하는 로직은 내 영역"이라는 명제를, leakage·refresh·RowHammer·ECC·aging 다섯 사례로 즉석에서 뒷받침할 수 있는가?